

1. RAG的优化

为了优化 RAG 系统，我们需要从以下几个方面入手：检索模块、生成模块。

根据之前的评估，我们可以有针对性的对某一方面进行优化。比如说如果 faithfulness、answer relevancy 比较低就要优化生成模块，而 context precision、context recall 比较低就要考虑优化检索模块了。

ps：本章我们只介绍一些常见的优化方法。

2. 检索模块优化

检索模块的主要任务是从外部知识库中查找与查询相关的文档。优化这一模块可以显著提升生成模型的上下文准确性。

2.1 优化分块

合理的分块一般是能保证块之间的差异性和块内部的一致性。但在实际应用中，知识库的文档多种多样，我们很难先验的完美分块。因此我们可以使用以下高级的分块方法：

- 对于中文可以使用jieba分词等方法取代按字符分割，避免一个词被从中间分割开。
- 自定义递归分割，根据知识库文档情况自定义分隔符达到更好的分割。
- 加入更多的元数据，比如说通过标题增强，即将每个章节的标题提取出来以一定格式拼接到对应章节的分块中，使每个分块带有标题的信息避免分块直接的割裂。
- 语义分割，通过计算向量化后的文本的相似度来进行语义层面的分割。
- 分块大小往往也是一个重要因素，在处理一些长篇幅文本时大的分块大小可以是主题更连贯，而对于一些较短的文本可能较小的分块大小更为合适。一般128的大小在一些测试中表现良好，但面对实际场景还是需要自己进行测试。

2.2 混合检索

什么是检索？简单来说，检索就是在大量信息中找到你需要的信息的过程。传统的检索方法，例如关键词搜索，是基于字面匹配的。也就是说，只有当你的搜索词和文档中包含相同的词或短语时，才能找到相关的文档。

传统的检索方法存在一些局限性：

- **词汇鸿沟 (Vocabulary Mismatch)：** 即使两个文本在语义上相关，但如果它们使用的词汇不同，就可能无法被检索到。例如，搜索“汽车”可能找不到包含“轿车”的文档。
- **无法捕捉语义信息：** 传统方法只能匹配字面上的词汇，无法理解文本的深层含义。例如，搜索“苹果公司”可能也会返回很多关于水果“苹果”的信息。

2.2.1 稠密检索

稠密检索的核心思想是将文本（包括查询和文档）转换成向量表示，这些向量捕捉了文本的语义信息。然后在向量空间中进行相似度比较，找到与查询向量最接近的文档向量。

核心思想： 将文本映射到向量空间，使得语义上相似的文本在向量空间中的距离也相近。

2.2.1.1 稠密检索的关键步骤

- **文本表示：** 使用深度学习模型（例如Sentence-BERT、Universal Sentence Encoder等）将文本转换为高维向量。这些向量被称为“稠密向量”或“嵌入向量”。
- **向量相似度计算：** 使用余弦相似度、点积或欧几里得距离等方法计算查询向量和文档向量之间的相似度。
- **检索：** 根据相似度得分对文档进行排序，返回最相关的文档。

2.2.1.2 举例说明

假设我们有以下两个文本：

- 文本1：“The cat sat on the mat.”（猫坐在垫子上。）
- 文本2：“A feline was resting on a rug.”（一只猫科动物在毯子上休息。）

使用传统的关键词检索，如果搜索“cat”和“mat”，则文本1会被检索到，而文本2则不会，因为它们没有共享相同的词汇。

但是，使用稠密检索，由于这两个文本在语义上非常相似，它们的向量表示也会非常接近，因此即使搜索“cat”和“mat”，文本2也很可能被检索到。

2.2.1.3 ANN（近似最近邻搜索）

但是由于文档库通常非常庞大，对每个查询都进行一次完整的向量比较（即暴力检索）是不切实际的，效率非常低下。因此，人们发明了近似最近邻搜索（ANN）算法。

核心思想： 在牺牲一定的精度（即不一定能找到真正的最近邻，与KNN不同）的前提下，大幅提高搜索速度。

常见算法：

- **局部敏感哈希（LSH）：** 将高维向量哈希到低维空间，使得相似的向量更有可能被哈希到同一个桶中。
- **向量量化（Vector Quantization, VQ）：** 将向量空间划分为若干个区域（或称为簇），每个区域用一个代表向量（即质心）表示。搜索时，只需比较查询向量与少量几个区域的质心，从而缩小搜索范围。
- **基于图的方法（Graph-based methods）：** 构建一个向量图，其中每个向量是一个节点，相似的向量之间有边相连。搜索时，从图中的某个节点出发，沿着边遍历，直到找到最近邻。例如HNSW (Hierarchical Navigable Small Worlds)。
- **倒排文件索引（IVF）：** 将向量聚类，然后构建倒排索引，加速查找过程。

2.2.2 稀疏检索

稀疏检索（Sparse Retrieval）将文本表示成高维稀疏向量，其中向量的每个维度对应一个词或短语，维度上的值表示该词在文本中的重要性。由于文本中出现的词语数量相对于词汇表来说非常少，因此向量中大部分维度上的值为0，故称为“稀疏向量”。

2.2.2.1 核心思想

基于词汇匹配，即通过比较查询和文档中是否包含相同的词或短语来进行检索。它不直接捕捉文本的语义信息，而是依赖于词汇的字面匹配。

2.2.2.2 特点

基于词汇匹配： 只考虑文本中出现的词语，不考虑词语之间的语义关系。

高维稀疏向量： 向量维度高，但大部分值为0，存储和计算效率较高。

可解释性强： 检索结果的原因比较清晰，可以很容易地看到哪些关键词匹配上了。

2.2.2.3 倒排索引 (Inverted Index)

讲倒排索引之前我们先看一下正向索引，正向索引是最直接的文档组织方式。它记录了每个文档中包含的所有词，并对每个文档建立索引。

假设我们有以下三个文档：

- 文档1：“我” “喜欢” “吃” “苹果”
- 文档2：“苹果” “是” “水果”
- 文档3：“苹果” “手机” “不错”

正向索引可以表示为：

文档编号	文档内容
1	“我” “喜欢” “吃” “苹果”
2	“苹果” “是” “水果”
3	“苹果” “手机” “不错”

在正向索引中，搜索“苹果”时，系统必须遍历所有文档才能确定哪些文档包含这个词。这在文档数量很多时，效率不高。

倒排索引记录每个词在哪些文档中出现，适合快速查找包含特定词的文档。倒排索引本质上是将每个词映射到包含该词的文档编号的列表中。

词	文档编号
“我”	1
“喜欢”	1
“吃”	1

词	文档编号
“苹果”	1、2、3
“水果”	2
“手机”	3
“是”	2
“不错”	3

在倒排索引中，搜索“苹果”时，系统可以直接查看“苹果”的倒排列表，快速得知它出现在文档1、文档2和文档3中。

简单来说正向索引是给定文档 -> 找词，倒排索引是给定词 -> 找文档。

对于向量数据库，倒排索引就是通过聚类划分为多个区域，每个向量和所有中心点向量对比归入不同的簇。

2.2.2.4 TF-IDF (Term Frequency-Inverse Document Frequency, 词频-逆文档频率)

这是一种衡量词重要性的算法，结合了词频和逆文档频率的思想。它不仅考虑一个词在文档中的出现次数，还考虑其在整个文档集合中的稀有程度。常见词（如“the”、“is”）会被赋予较低的权重，而不常见但关键的词则具有更高的权重。

- TF 词频(Term Frequency)指的是某个词在文档中出现的频率。

$$TF = \frac{\text{某个词在当前文档中出现的次数}}{\text{当前文档的总词数}}$$

- IDF 逆向文件频率(Inverse Document Frequency)衡量的是某个词在整个语料库中的普遍程度。词越常见，IDF值越低。

$$IDF = \log\left(\frac{\text{总文档数} + 1}{\text{包含该词的文档数} + 1}\right) + 1$$

分母的加1避免IDF值为零或负数，并在对数运算后加1，确保IDF值始终为正。

TF-IDF的计算公式如下：

$$TF - IDF = TF \times IDF$$

还是用上面例子进行计算：

对于文档1，每个词都只出现了一次因此对应的词频TF都是1/4；

同理对于文档2、3每个词词频TF都是1/3；

计算每个词在几个文档中出现过，然后代入IDF公式（假设底数为自然对数ln）：“我”“喜欢”“吃”“手机”“不错”“是”“水果”都只出现在一份文档中

$$IDF = \ln\left(\frac{3+1}{1+1}\right) + 1 \approx 1.69, \text{“苹果”出现在三份文档中}$$

$$IDF = \ln\left(\frac{3+1}{3+1}\right) + 1 = 1.$$

因此在文档1中：“我”、“喜欢”、“吃”的TF-IDF是 $0.25 \times 1.69 = 0.4225$ ，“苹果”是 $0.25 \times 1 = 0.25$ 。这表示“苹果”不是一个很好的区分词。

使用代码计算：

```
1 # TF-IDF
2
3 import math
4 from collections import Counter
5
6 # 定义文档集
7 documents = [
8     "我 喜欢 吃 苹果",
9     "苹果 是 水果",
10    "苹果 手机 不错"
11 ]
12
13
14 # 预处理：将每个文档分词
15 def preprocess(document):
16     return document.split()
17
18
19 # 计算词频 (TF)
20 def compute_tf(document):
21     tf_dict = {}
22     word_count = Counter(document)
23     total_words = len(document)
24     for word, count in word_count.items():
25         tf_dict[word] = count / total_words
26     return tf_dict
```

```

27
28
29 # 计算逆文档频率 (IDF)
30 def compute_idf(documents):
31     idf_dict = {}
32     total_docs = len(documents)
33     all_words = set([word for doc in documents for word in
doc])
34
35     for word in all_words:
36         count = sum(1 for doc in documents if word in doc)
37         # total_docs + 1 是为了使结果和sklearn对应做的光滑
38         idf_dict[word] = math.log((total_docs + 1) / (count +
1)) + 1
39     return idf_dict
40
41
42 # 计算TF-IDF
43 def compute_tfidf(tf, idf):
44     tfidf = {}
45     for word, tf_value in tf.items():
46         tfidf[word] = tf_value * idf[word]
47     return tfidf
48
49
50 # 主程序
51 processed_docs = [preprocess(doc) for doc in documents]
52 tf_list = [compute_tf(doc) for doc in processed_docs]
53 idf = compute_idf(processed_docs)
54
55 # 输出每个文档的TF-IDF值，带L2标准化
56 for i, tf in enumerate(tf_list):
57     tfidf = compute_tfidf(tf, idf)
58
59     # 计算L2范数
60     norm_ = math.sqrt(sum(value ** 2 for value in
tfidf.values()))
61
62     print(f"文档 {i + 1} 的 L2 标准化后的 TF-IDF 值: ")
63
64     # 输出L2标准化后的TF-IDF值

```

```

65     for word, value in tfidf.items():
66         normalized_value = value / norm_ if norm_ > 0 else 0
        # 防止除以0,L2标准化
67         # normalized_value = value # 防止除以0, 非L2标准化
68         print(f"{word}: {normalized_value:.4f}")

```

使用sklearn, sklearn的TfidfVectorizer 默认使用了一些平滑和归一化策略, 以及不同的 IDF 计算公式, 导致结果有所不同。

例如 smooth_idf=False: 这将禁用 IDF 的平滑处理。

sublinear_tf=False: 这将禁用子线性 TF 缩放。

norm=None: 这禁用了归一化 (L1或者L2) 。

```

1  # 使用sklearn
2  from sklearn.feature_extraction.text import TfidfVectorizer
3
4  documents = [
5      "我 喜欢 吃 苹果",
6      "苹果 是 水果",
7      "苹果 手机 不错"
8  ]
9
10 #初始化
11 # 需要设置 token_pattern 否则短的词将被丢掉
12 vector = TfidfVectorizer(token_pattern=r"(?u)\b\w+\b")
13
14 #tf-idf计算
15 tfidf = vector.fit_transform(documents)
16
17 print(vector.vocabulary_)
18 #直接打印, 得到的是一个稀疏矩阵, 第1位表示文档编号, 第2位代表词的编号
19 print(tfidf.toarray())

```

2.2.2.5 BM25 (Best Matching 25)

一种改进的 TF-IDF 算法, 是稀疏检索的主流模型之一。BM25 通过引入非线性词频饱和函数和文档长度归一化来进一步提升检索效果, 能够有效减少长文档中高频词的过度影响。

主要解决了以下问题：对于TF-IDF算法，TF越大TF-IDF值就越大，如果一个文档很长那么词频TF就会很大导致长文档比短文档容易有更高的TF值，影响公平性。

BM25在此基础上加入了参数k惩罚长文档，随着TF 的逐步加大，TF-IDF会趋于一个数值。然后我们得到了以下公式

$$BM25(t, d) = \sum_{t \in q} IDF(t) \cdot \frac{TF(t, d) \cdot (k_1 + 1)}{TF(t, d) + k_1 \cdot \frac{|d|}{avgdl}}$$

其中k是可调节参数用于压制TF的线性增长， avgdl 是文档集的平均长度； |d| 是文档的长度。

但是对于某些语料库来说文档的长度非常重要，而另一些语料库文档的长度则根本不重要。于是加入一个可调节参数b，当b等于0时表示我们并不重视文档的长度，反正非常重视文档长度。

$$BM25(t, d) = \sum_{t \in q} IDF(t) \cdot \frac{TF(t, d) \cdot (k_1 + 1)}{TF(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{avgdl}\right)}$$

其中： k_1 和b是可调节参数，通常 k_1 的取值在1.2到2.0之间， b取值为0.75； avgdl 是文档集的平均长度； |d| 是文档的长度。注意这里的TF是词在文档中出现的次数

BM25 的 IDF 部分方程与 TF-IDF 稍有不同。BM25 的 IDF 公式定义如下：

$$IDF(x) = \log\left(\frac{N - DF(x) + 0.5}{DF(x) + 0.5}\right)$$

其中， N 是语料库中的文档总数， DF 是包含关键词的文档数量。0.5也是为了光滑。

```

1  # BM25
2  import math
3  from collections import Counter
4
5  # BM25参数
6  k1 = 1.5
7  b = 0.75
8
9  # 文档集
10 documents = [
11     "我 喜欢 吃 苹果",
12     "苹果 是 水果",

```

```

13     "苹果 手机 不错"
14 ]
15
16
17 # 预处理：将每个文档分词
18 def preprocess(document):
19     return document.split()
20
21
22 # 计算词频 (TF)
23 def compute_tf(document):
24     word_count = Counter(document)
25     return word_count
26
27
28 # 计算逆文档频率 (IDF)
29 def compute_idf(documents):
30     total_docs = len(documents)
31     idf_dict = {}
32     negative_idfs = []
33     idf_sum = 0
34     epsilon = 0.25
35     all_words = set(word for doc in documents for word in doc)
36
37     for word in all_words:
38         doc_count = sum(1 for doc in documents if word in doc)
39         idf_dict[word] = math.log((total_docs - doc_count +
40 0.5) / (doc_count + 0.5))
41
42     # 以下部分是为了和库函数结果保持一致，实际作用就是将小于0的idf变成大于0的。
43     idf_sum += idf_dict[word]
44     if idf_dict[word] < 0:
45         negative_idfs.append(word)
46     average_idf = idf_sum / len(idf_dict)
47     eps = epsilon * average_idf
48     for word in negative_idfs:
49         idf_dict[word] = eps
50     return idf_dict
51
52 # 计算BM25

```

```

52 def compute_bm25(tf, idf, doc_len, avgd1):
53     bm25 = {}
54     for word, tf_value in tf.items():
55         numerator = tf_value * (k1 + 1)
56         denominator = tf_value + k1 * (1 - b + b * (doc_len /
avgd1))
57         bm25[word] = idf[word] * (numerator / denominator)
58     return bm25
59
60
61 # 主程序
62 processed_docs = [preprocess(doc) for doc in documents]
63 tf_list = [compute_tf(doc) for doc in processed_docs]
64 idf = compute_idf(processed_docs)
65
66 # 计算文档的平均长度
67 doc_lengths = [len(doc) for doc in processed_docs]
68 avgd1 = sum(doc_lengths) / len(documents)
69
70 # 输出每个文档的BM25值
71 for i, tf in enumerate(tf_list):
72
73     bm25 = compute_bm25(tf, idf, len(processed_docs[i]),
avgd1)
74     print(f"文档 {i + 1} 的 BM25 值: ")
75     for word, value in bm25.items():
76         print(f"{word}: {value:.4f}")

```

使用库

```

1 # pip install rank-bm25
2 from rank_bm25 import BM25Okapi
3
4 # 文档集
5 documents = [
6     "我 喜欢 吃 苹果",
7     "苹果 是 水果",
8     "苹果 手机 不错"
9 ]
10
11 # 预处理：将每个文档分词

```

```

12 tokenized_documents = [doc.split() for doc in documents]
13
14 # 初始化BM25
15 bm25 = BM25Okapi(tokenized_documents)
16
17 # 打印每篇文档的BM25评分（以 "苹果" 为例）
18 query = "我 喜欢 苹果".split()
19 doc_scores = bm25.get_scores(query)
20
21 print(f"{query}在每篇文档的BM25评分: ")
22 for idx, score in enumerate(doc_scores):
23     print(f"文档 {idx + 1} 的评分: {score:.4f}")

```

对于一句话的检索来说就是将每个词在每个文档中的BM25值加起来，就得到最后的相关性。

2.2.3 混合检索（多路召回）

可以将这两种方法的结果进行融合，提升检索的全面性。

使用langchain实现向量和BM25的混合检索。使用weights控制两个检索器的权重。

```

1 # 混合检索
2 import jieba
3 from rank_bm25 import BM25Okapi
4 from langchain.retrievers import EnsembleRetriever
5 from langchain_community.vectorstores import FAISS
6 from langchain_community.retrievers import BM25Retriever
7 from langchain_community.embeddings import
  HuggingFaceEmbeddings
8
9 # 定义文档列表
10 documents = [
11     "Python 是一种广泛使用的编程语言。",
12     "JavaScript 被广泛用于 web 开发。",
13     "机器学习是人工智能的一个分支。",
14     "FAISS 是一个高效的向量相似性搜索库。",
15     "BM25 是一种常用于信息检索的评分函数。"
16 ]
17
18 # 将文档转换为 BM25 格式

```

```

19 tokenized_corpus = [list(jieba.cut(doc.lower())) for doc in
documents]
20 print(tokenized_corpus)
21 bm25 = BM25Okapi(tokenized_corpus)
22
23 # 初始化嵌入模型
24 embedding = HuggingFaceEmbeddings(model_name='models/AI-
ModelScope/bge-large-zh-v1___5')
25
26 # 构建 FAISS 向量存储和对应的检索器
27 vs = FAISS.from_texts(documents, embedding)
28 # 使用 FAISS 进行相似度检索
29 faiss_retriever = vs.as_retriever(search_kwargs={"k": 2}) # 将
FAISS 向量存储包装为检索器对象
30
31 # 初始化 BM25 检索器
32 bm25_retriever = BM25Retriever.from_texts(documents)
33 bm25_retriever.k = 2 # 设置 BM25 检索器返回的文档数量
34 bm25_retriever.vectorizer = bm25
35
36 # 构建混合检索器
37 ensemble_retriever = EnsembleRetriever(
38     retrievers=[bm25_retriever, faiss_retriever], weights=
[0.5, 0.5]
39 )
40
41 # 使用混合检索器进行检索
42 docs = ensemble_retriever.invoke("python")
43 page_contents = [doc.page_content for doc in docs]
44 print(page_contents)

```

2.3 优化查询

2.3.1 问题重写

通常用户的输入是比较模糊的，直接使用原始输入进行检索往往得不到很好的结果，问题重写是指在接收到用户的模糊输入后，对其进行结构化或补充更多信息，从而提升检索或问题回答的准确性。

2.3.2 后退提示

生成一个“退后”的问题，在使用检索时，将同时使用“退后”问题和原始问题进行检索。例如，当用户询问“Estella Leopold 在 1954 年8月至1954 年11月期间去了哪所学校？”时，由于问题中的时间范围过于具体。大模型比较难回答，此时可以将问题退化为“Estella Leopold 的教育经历是什么？”，后退提示会先询问“教育史”，这是一个包含原始问题的高级概念。这样更容易得到更为广泛的信息使大模型的推理更为准确。

论文 <https://ar5iv.labs.arxiv.org/html/2310.06117>。

2.3.3 多查询

基于原始问题，提示LLM从不同角度产生多个新问题或者子问题，并使用每一个新问题进行检索，特别是在问题模糊或主题广泛的情况下，多查询策略可以提升命中率。

2.4 Rerank

Rerank（重新排序）是对检索得到的候选文档进行排序或筛选的一个步骤，目的是确保生成模块输入的文档更加相关，将最相似的文档放在顶部，以提高生成结果的质量。

在初始检索阶段，通常会使用向量相似度（如余弦相似度）对文档和查询进行匹配。在Rerank阶段，可以使用更加复杂的向量相似度算法来精细化文档的排名。例如：

- BERT-based Rerank：可以通过BERT模型对候选文档和查询进行深层次的语义匹配，从而精确排序。这种方法比简单的余弦相似度能捕捉到更多语义上的细微差异。
- Cross-encoder：利用交叉编码器（cross-encoder）对查询和文档进行联合编码，然后基于输出的分数来进行排序。

```
1 # rerank
2
3 loader = TextLoader("黑悟空.txt", encoding='utf-8')
4 docs = loader.load()
5
6 # 把文本分割成 200 字一组的切片
7 text_splitter = RecursiveCharacterTextSplitter(chunk_size=200,
8 chunk_overlap=20)
9 chunks = text_splitter.split_documents(docs)
```

```

9
10 # 初始化嵌入模型
11 embedding = HuggingFaceEmbeddings(model_name='models/AI-
    ModelScope/bge-large-zh-v1____5')
12
13 # 初次检索
14 # 构建 FAISS 向量存储和对应的 retriever
15 vs = FAISS.from_documents(chunks, embedding)
16 context = vs.similarity_search("黑熊精自称为? ")
17 print("rerank前: ", context)
18
19 # rerank 重排序
20 import torch
21 from transformers import AutoModelForSequenceClassification,
    AutoTokenizer
22 device = "cuda" if torch.cuda.is_available() else "cpu"
23
24 pairs = [["黑熊精自称为?", c.page_content] for c in context]
25
26 tokenizer = AutoTokenizer.from_pretrained('models/AI-
    ModelScope/bge-reranker-v2-m3')
27 model =
    AutoModelForSequenceClassification.from_pretrained('models/AI-
    ModelScope/bge-reranker-v2-m3').to(device)
28 model.eval()
29
30 with torch.no_grad():
31     inputs = tokenizer(pairs, padding=True, truncation=True,
        return_tensors='pt', max_length=512).to(device)
32     scores = model(**inputs, return_dict=True).logits.view(-1,
        ).float()
33     res = sorted([(pair[:,1], score) for pair, score in
        zip(pairs, scores)], key=lambda x: x[1], reverse=True)
34     print("rerank后: ", res)

```

3. 生成模块优化

生成模块的任务是基于检索到的文档，生成连贯且有用的回答。通过优化生成模型，可以提升输出的质量。

3.1 使用更好的大模型

最直接暴力的优化方法，等号的大模型有更好的推理效果可以在复杂的检索内容中找到答案。

3.2 生成策略优化 temperature & Top_p

低的temperature和Top_p往往得到更稳定准确的结果。

3.3 使用更好的 prompt

更好的prompt可以引导大模型的思考，之前提到的对于查询的优化都是基于prompt。

华清远见/元宇宙实验中心
yyzlab.com.cn